

Frameworks

Prof. Sandro Laudares

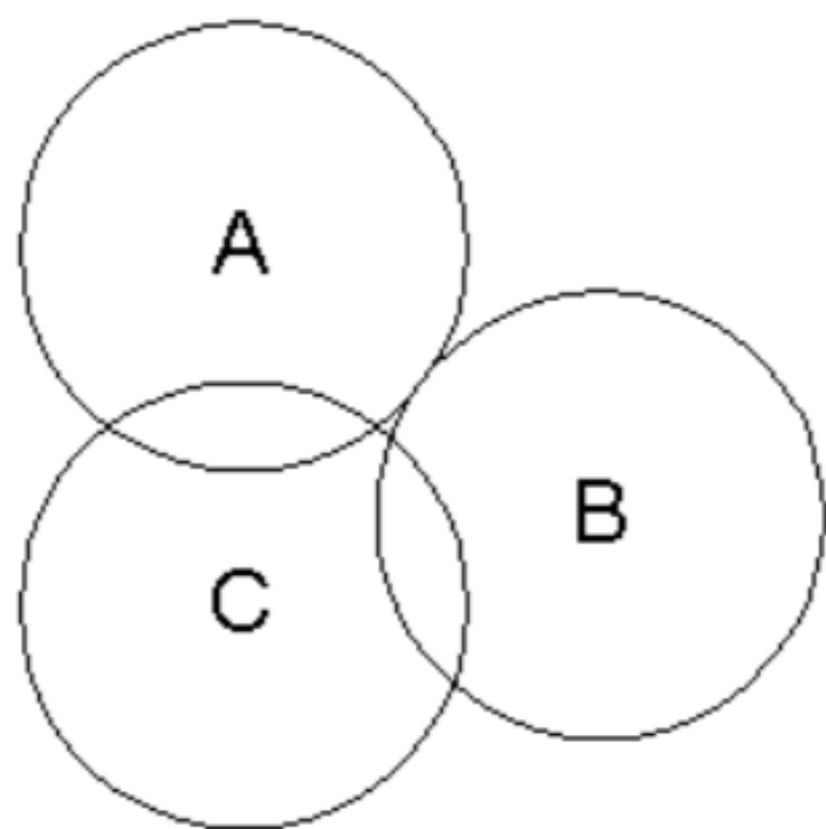
- Programar é difícil
- Onde está a maior dificuldade?
 - "Interface design and functional factoring constitute the key intellectual content of software and is far more difficult to create or re-create than code" (Peter Deutsch)
- Mas nossos programadores são mortais
 - "It shouldn't take a good programmer to build a good program"
- Solução: Temos que fornecer formas de re-uso que vão além de código: re-uso de análise, design, código.
 - Framework orientado a objeto

O que é um Framework?

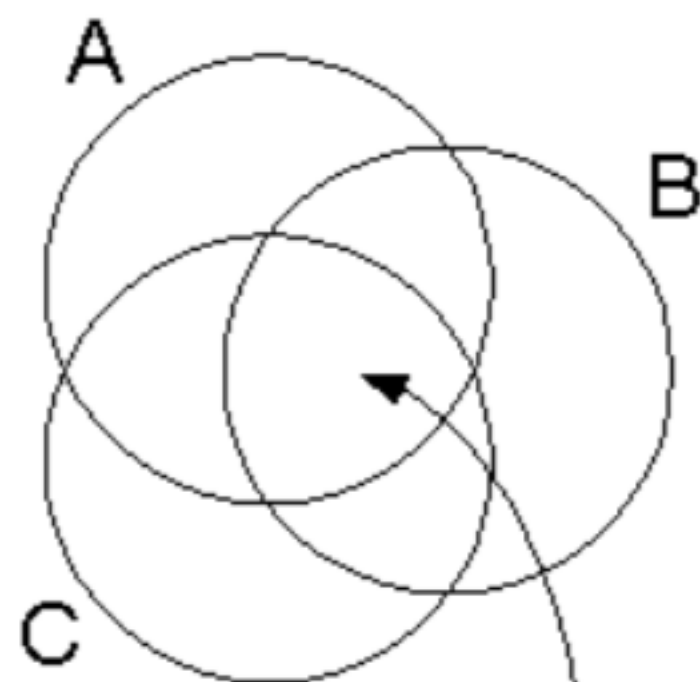
- Um framework captura a funcionalidade comum a várias aplicações
- As aplicações devem ter **algo razoavelmente grande em comum**: pertencem a um mesmo domínio de problema

Definições de framework

- Johnson-1:** "A framework is a set of classes that embodies an abstract design for solutions to a family of related problems"
- Johnson-2:** "A framework is a set of objects that collaborate to carry out a set of responsibilities for an application subsystem domain"
- Johnson-3:** "A framework is a set of abstract classes and the way objects in those classes collaborate"
- Gamma:** "A framework is a set of cooperating classes that make up a reusable design for a specific class of software"
- Taligent:** "A framework is an extensible set of object-oriented classes that are integrated to execute well-defined sets of computing behavior"
- Govoni:** "A framework is an abstracted collection of classes, interfaces, and patterns dedicated to solving a class of problems through a flexible and extensible architecture"
- Rogers:** "A framework is a class library that captures patterns of interaction between objects. A framework consists of a suite of concrete and abstract classes, explicitly designed to be used together"



Impossível criar
Framework



Interseção grande
Possível criar Framework

"Um framework provê uma **solução para uma família de problemas** semelhantes, ... Usando um **conjunto de classes e interfaces** que mostra como decompor a família de problemas, ...

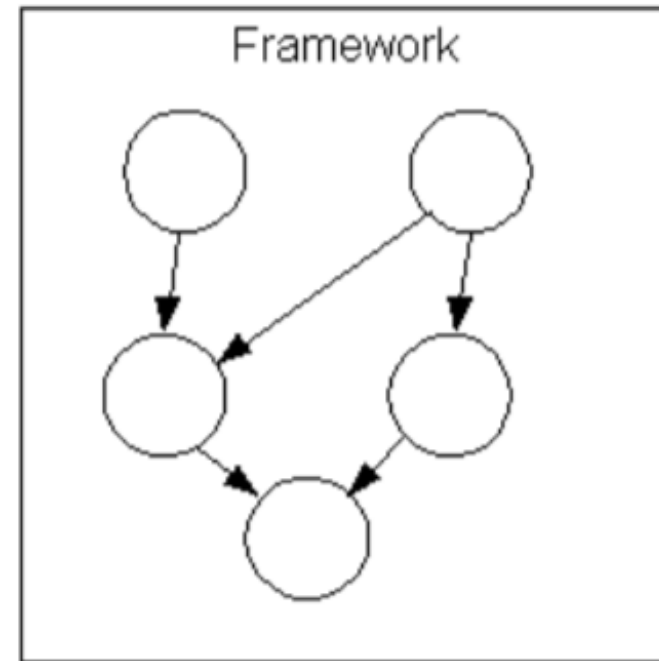
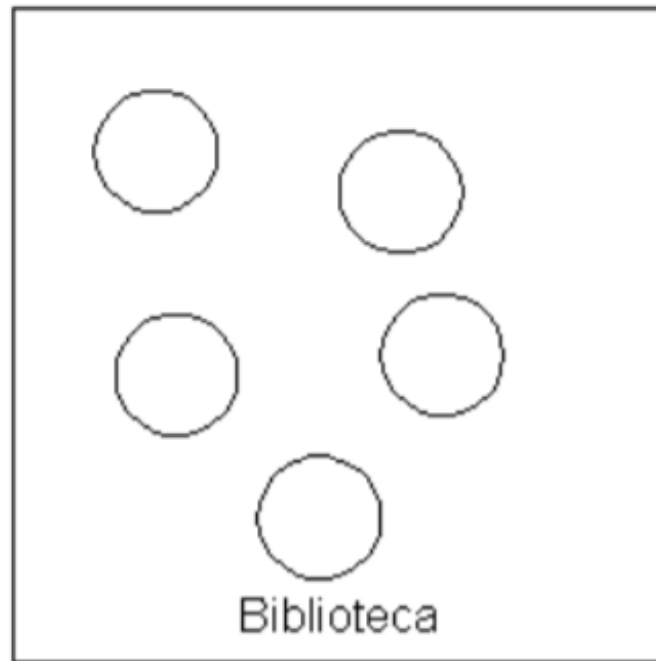
E como objetos dessas classes **colaboram** para cumprir suas responsabilidades, ...

O conjunto de classes deve ser **flexível e extensível** para permitir a construção de várias aplicações com pouco esforço, especificando apenas as particularidades de cada aplicação"

Framework X Biblioteca de Classes

Num framework, as dependências/colaborações estão embutidas (wired interconnections)

Com biblioteca, as aplicações criam as colaborações



Vê-se portanto que um framework impõe um **modelo de colaboração** (o resultado da análise e design) ao qual você deve se adaptar

•**Frontend development:** A frontend developer is, in large part, responsible for what a user sees. The 3 main tools a frontend developer works with are HTML, CSS, and JavaScript. Their primary focus should be to create a great user experience and ensure that the design and layout of the website or web app is cohesive throughout.

X

•**Backend development:** A backend developer on the other hand works on the nuts and bolts of what goes on behind the scenes. Backend developers work with a wide range of libraries, APIs, web services, etc. They are responsible for the implementation of database systems, ensuring proper communication between various web services, generating backend functionality, and more.

Preocupações do desenvolvedor front-end

The main concerns of a front end developer include:

- user interface (UI),
- responsive web design
- accessibility and
- compatibility.

O que considerar ao selecionar um framework?

- **Learning curve:** Each framework has its differences in terms of naming conventions, directory structures, etc. If you've never used a framework before this part of the learning curve is inevitable. However, if you're familiar with a particular programming language already, you may want to stick to a framework which utilizes that language for time's sake. On the other hand if you aren't yet familiar with a specific language, the Django framework which uses Python is a good place to start as it is fairly straightforward.
- **Documentation/community:** If you don't have any previous experience with a backend framework it may be favorable to opt for a framework which has extensive documentation and/or a large community. Therefore, if you ever get stuck you can search for the proper documentation or ask someone in the community for help.
- **Libraries:** Some frameworks offer more flexibility and choice in terms of the libraries offered. This allows developers to implement or configure a library in order to perform a certain task that would otherwise be time consuming or not possible.
- **Security:** Security is very broad topic and depending on the application you're building, you may require certain security features which some backend frameworks perform better than others. It's also important that the framework you choose is regularly updated to ensure that it is equipped to protect itself against common and new attacks.
- **Scalability:** If you're building an app for the masses, you'll want to ensure that your framework provides the ability to scale with you. Its caching capabilities will play a big part in this and will determine how easy it is to scale your application

Front-end JavaScript Frameworks

Frontend development refers to the process of creating the interactive side or interface of a website. This is the side users see and interact with while browsing through a website. It is also sometimes called “client-side” or “browser side”.

A static website is what you can refer to as a front-end application. Its content is embedded in the frontend code which doesn't change much over time. This type of development is ideal for developing static websites for businesses, portfolio, profiles, etc.

A framework which is used for developing the front-end of a web application is called a **front-end framework**. JavaScript, CSS, and HTML are some of the [most popular programming languages for front-end development](#).

Recently IEEE spectrum comes with the ranking sheet of top programming language 2020 according to their popularity – **Programming Languages Popularity**

Language Rank	Types	Spectrum Ranking
1. Python	 	100.0
2. C	  	99.7
3. Java	  	99.5
4. C++	  	97.1
5. C#	  	87.7
6. R		87.7
7. JavaScript	 	85.6
8. PHP		81.2
9. Go	 	75.1
10. Swift	 	73.7

- Frameworks podem ser entendidos como uma técnica de reuso
- Tem sido utilizados há um bom tempo mas muitos projetos de Frameworks não tem sido bem sucedidos
- Há também a falta de uma metodologia orientada a objetos que indique como usar um framework

A maioria dos frameworks comerciais tratam de interfaces, persistência e distribuição

A maioria dos frameworks para aplicações específicas são proprietários

Exemplos: frameworks back-end

Hibernate – framework para mapeamento objetorelacional <http://hibernate.org/>

Struts – framework para desenvolvimento de camada controladora em uma estrutura Model 2(similar ao MVC) <http://struts.apache.org/>

Ruby on Rails- framework que facilita o desenvolvimento de aplicativos web orientados a banco de dados (usa também o MVC) <http://rubyonrails.org/>

Um framework é um projeto de um sistema, reutilizável que descreve como o sistema é decomposto em um conjunto de objetos que interagem

Às vezes o sistema é uma aplicação inteira ou um subsistema

O framework descreve:

- os objetos componentes e suas interações
- a interface de cada objeto e o fluxo de controle entre eles
- como as responsabilidades do sistema são mapeadas nos objetos

A parte mais importante de um framework é o modo em que ele é dividido em componentes. Importante:

- o reuso das interfaces internas do sistema;
- e como as funções são divididas em seus componentes.

Um framework possui um modelo colaborativo e um padrão de interação.

Frameworks usufruem das 3 principais características de linguagens O.O:

- Abstração de dados
- Polimorfismo
- Herança

Classificação de frameworks quanto ao escopo

Frameworks para infraestrutura – facilitam o desenvolvimento de software básico portátil e eficiente tais como S.O.'s, infraestrutura para comunicação, interface com o usuário e processamento de linguagem.

Frameworks de integração – são usados comumente para integrar aplicações e componentes distribuídos.

Frameworks de integração Middleware são usados para que os desenvolvedores modularizem, reutilizem, estendam sua infraestrutura de software para trabalhar sem emendas em um ambiente distribuído. Exemplos: ORB frameworks, message-oriented middleware e bancos de dados transacionais.

Enterprise Application Frameworks – frameworks que tratam de vastos domínios de aplicação. São caros de desenvolver e apesar de agilizarem o desenvolvimento de aplicativos para o usuário final, seu retorno é pequeno quanto comparado a frameworks de infra-estrutura e middleware de integração.

White-box frameworks – dependem fortemente de características de linguagens O.O como herança e ligação dinâmica para alcançar extensibilidade. A funcionalidade existente é reutilizada e estendida via:

- (1) herança de classes base;
- (2) sobrescrita de métodos hook usando padrões como o template method

*** O uso de frameworks do tipo white-box requer conhecimento intimo da estrutura interna do framework, em particular da estrutura de heranças que define os componentes

Black-box frameworks – suporta extensibilidade pela definição de interfaces que podem ser acopladas no framework via composição de objetos:

- (1) definição de componentes que respeitam a interface
- (2) integração no framework usando padrões como o Strategy

*** Black-box frameworks são estruturados usando composição e delegação. Logo, black-box frameworks são mais fáceis de usar, já que requerem menos conhecimento da estrutura interna do framework. São mais difíceis de desenvolver já que é necessário desenvolver interfaces e hooks que prevejam a grande variedade de casos de uso

Manutenibilidade

- Os requisitos de frameworks também mudam
- Modificações e adaptações podem ocorrer em nível funcional (a funcionalidade não satisfaz os requisitos dos desenvolvedores) e nível não-funcional(aspectos qualitativos com reusabilidade e portabilidade)
- Os desenvolvedores da aplicação dependem dos desenvolvedores dos frameworks

Manutenibilidade

Validação e remoção de falhas:

- Difícil validar componentes abstratos
- Dificuldade de depuração devido a inversão de controle o fluxo alternar entre os componentes da infra-estrutura do framework independente de aplicações e as callbacks específicas da aplicação

Eficiência:

– a generalidade e flexibilidade de um framework em geral reduzem a eficiência e isto acarreta em:

- (1) aumento no armazenamento (necessidade de ponteiros para tabelas virtuais)
- (2) degradação do desempenho (overhead para invocar métodos ligados dinamicamente)
- (3) inability para armazenar objetos em memória compartilhada



Como usar frameworks mais específicos

- O desenvolvimento baseado em um framework requer que a aplicação se encaixe no modelo do framework.
- O projeto de uma aplicação baseada em um framework deve ser criado a partir do projeto do framework.
- Uma aplicação baseada em framework tem 3 partes:
 - O framework
 - As subclasses concretas que herdam das classes abstratas do framework
 - Outros elementos

- Modos de utilizar um framework em função da dificuldade:
 - Conectar componentes existentes
 - Definir novas subclasses concretas
 - Estende-lo modificando as subclasses abstratas, criando novas operações e adicionando variáveis membro



Mais
difícil

- Uso por conexão de componentes:
 - Não altera o framework
 - Não é preciso criar subclasses concretas
 - Reutiliza as interfaces e regras do framework para conectar componentes
 - Funciona como Lego

- Uso por definição de subclasses
 - Requer a criação de subclasses concretas
 - Requer mais conhecimento sobre as classes abstratas
 - O programador deve conhecer as interfaces do framework em detalhes

- Uso por extensão e modificação das classes abstratas
 - Modo mais difícil de usar um framework
 - Mais poderoso entretanto
 - Mudanças a classes abstratas podem quebrar a consistência de classes concretas

Como aprender a usar um framework

Frameworks são mais difíceis de aprender que uma biblioteca

- As classes mais importantes são abstratas o que torna o processo de aprendizado mais difícil
- Frameworks são mais fáceis de aprender se existe uma boa documentação
- **A melhor forma de aprender um framework é por exemplos**
Exemplos são concretos e mais fáceis de compreender que o framework como um todo.
- Exemplos resolvem problemas específicos e você pode estudar sua execução para aprender o fluxo de controle dentro de um framework

- Etapas no projeto de um framework:
 1. Análise do domínio
 2. Coleta de exemplos
 3. Geração de uma versão whitebox que implementa os exemplos
 4. Uso do framework para construir aplicações
 5. Determinação dos pontos fracos que são as partes difíceis de modificar
 6. Melhorias no framework tornando-o mais blackbox
 7. Estabilização do framework
 8. Produção

- É difícil criar frameworks em prazos devido a necessidade de se conhecer em profundidade o domínio da aplicação e as diversas iterações
- O projeto de um framework não deve jamais estar no caminho crítico de um projeto importante

Considerações

- Somente o uso do framework é que vai apontar suas fraquezas
- Isso leva a um processo de especialização que ocorre diversas vezes
- Logo, o desenvolvimento de um framework pode requer um número indeterminado de iterações do processo de desenvolvimento

Referencias

Mohamed E. Fayad (Editor), Douglas C. Schmidt (Editor), Ralph E. Johnson (Editor).
Building Application Frameworks: Object-Oriented Foundations of Framework Design.
Wiley; 1 edition (September 13, 1999). Capítulos 1, 15 e 16.

<https://blog.mastertech.com.br/tecnologia/frameworks-de-desenvolvimento/>

<https://hackr.io/blog/best-javascript-frameworks>